The code for the new skin component looks like this:

```
<?xml version="1.0" encoding="utf-8"?>
<s:SparkSkin xmlns:fx="http://ns.adobe.com/mxml/2009"
xmlns:s="library://ns.adobe.com/flex/spark"
xmlns:mx="library://ns.adobe.com/flex/mx">
<fx:Declarations>
<!-- Place non-visual elements
(e.g., services, value objects) here -->
</fx:Declarations>
</s:SparkSkin>
```

The new skin component's code looks like that of any other custom component and can contain instances of any Flex component. Its difference lies in its superclass. Because its root element is <s:SparkSkin>, its purpose is to implement the component's visual design.

After creating the new skin component, you should associate it with the Spark component it's designed to modify. You create the association with the [HostComponent] metadata tag. This tag takes as its only attribute a string that describes the fully qualified package and name of the component:

```
[HostComponent("spark.components.Button")]
```

As with the [Event] metadata tag, [HostComponent] is placed inside the <fx:Metadata> element. Follow these steps to add the component association to the new custom skin:

1. Add a blank line under the <s:SparkSkin> starting tag.
2. Add the following code to the component:

```
<fx:Metadata>
[HostComponent("spark.components.Application")]
<fx:Metadata>
```

In your skin component, you'll be able to refer to the host component's properties using ActionScript code and binding expressions. You refer to the host object with an id of host Component, and to its properties using dot notation. For example, in a custom skin for the Application component, you could output the application's current height and width to the screen with the following Label and binding expression:

```
<s:Label text="{hostComponent.width},{hostComponent.height}"/>
```

In the next step, you declare view states to match the host component's required skin states. You don't have to refer to these view states in the component's visual design. For example, the Spark Application component has two required states. If you want the application to look the same in all view